CONCEPTION DE SYSTÈMES INFORMATIQUES SÉCURITAIRES

LABORATOIRE 2 — ÉNONCÉ ET GRILLE D'ÉVALUATION

Attaques mémoire, outils de bas niveau et première défense par le typage (Version 1.1)

Période / Cours	Session d'Automne — Modèle de laboratoire pratique réutilisable
Échéances	**Démonstration :** Séance 2 (présence obligatoire de tous les membres). **Remise du rapport :** Vendredi suivant la séance 2, à 23 h 59.
Modalités	Pondération : 5% de la note finale. Équipe de 2 étudiants. Grille globale sur 100 points.
Usage autorisé	Les techniques offensives de ce laboratoire (débordement de tampon, shellcode, return-to-libc) ne s'appliquent qu'aux programmes fournis dans l'environnement isolé du cours (machine virtuelle ou conteneurs SEED). Toute action contre un système tiers est strictement interdite et traitée comme une inconduite. L'entente d'usage autorisé signée au laboratoire 1 demeure en vigueur.

RENSEIGNEMENTS SUR LE LABORATOIRE

Élément	Détail
Séance 1 : Shellcode & débordement	Disposition de la pile, développement du shellcode, débordement de tampon (Niveaux 1 à 3) et contre-mesures (dash, ASLR, StackGuard, pile non exécutable).
Séance 2 : Return-to-libc, défense & démo	Attaque return-to-libc, chaîne source-assembleur-code machine, réécriture en Ada et démonstration devant l'assistant.
Environnement de référence	Machine virtuelle SEED Ubuntu 20.04 (suite SEED 2.0) ou conteneurs Docker (requis pour l'extension Serveur). C et assembleur x86; `gcc`, `gdb`, `objdump`, `readelf`, `nasm`, `ld`, `execstack`, `sysctl`. Pour Ada : chaîne GNAT FSF et `gprbuild` (installés via Alire).
Thèmes SEED (fournis)	« Shellcode Development Lab », « Buffer Overflow Attack Lab (Set-UID Version) » et « Return-to-libc Attack Lab ». Extension obligatoire : « Buffer Overflow Attack Lab (Server Version) ». Dossiers `Labsetup` associés.
Préparation	Manuel W. Du, ch. 4 (débordement), 5 (return-to-libc) et 9 (shellcode). Introduction to Ada, ch. « Strongly typed language », « Records » et « Arrays ». Notes de cours sur la sécurité mémoire.
Composante CLO / Légué	**CLO 9 / PI 5.1 / GA 5 :** Relier source C, assembleur et code machine et analyser l'effet des options de compilation. Légué au projet final : corpus d'entrées mémoire et assertions vérifiables.

1. CONTEXTE ET OBJECTIFS

Les laboratoires du cours sont cumulatifs et convergent vers le projet final : chacun produit des modules, des propriétés de sécurité et un corpus d'entrées dangereuses réutilisés ultérieurement. Le laboratoire 2 étudie la deuxième grande classe de vulnérabilités : les erreurs de gestion de la mémoire en C. L'équipe reproduit un débordement de tampon détournant le flot de contrôle, développe un shellcode, monte une attaque *return-to-libc* sur pile non exécutable, évalue les contre-mesures (dash, ASLR, StackGuard, NX), relie le code C à l'assembleur/code machine, et met en œuvre une réécriture en Ada avec sous-types contraints pour lever une exception maîtrisée.

La démarche s'applique en quatre temps : observer l'attaque, remonter à la cause première, écrire l'hypothèse violée comme propriété explicite, puis vérifier que le code corrigé la respecte.

Objectifs d'apprentissage

À la fin de ce laboratoire, l'étudiant est capable de :

- Expliquer la disposition de la pile d'exécution et du cadre d'appel (tampon, pointeur de cadre sauvegardé, adresse de retour).
- Écrire et tester un shellcode fonctionnel (32 et 64 bits) de l'assembleur au code machine, en éliminant les octets nuls.
- Construire une entrée malveillante qui écrase l'adresse de retour et exécute le code injecté avec le privilège du programme.
- Activer/désactiver la protection de dash, l'ASLR, StackGuard et la pile NX, et documenter ce que chacune bloque ou ne bloque pas.
- Monter une attaque *return-to-libc* qui contourne la pile NX en réutilisant `system()` et la chaîne `"/bin/sh"`.
- Relier le code source C à l'assembleur et au code machine, et analyser l'effet des options de compilation (composante CLO 9).
- Réécrire la routine d'analyse d'entrée en Ada avec sous-types contraints, et observer le contrôle de contrainte lever un `Constraint_Error`.
- Écrire les hypothèses violées comme assertions vérifiables et constituer un corpus d'entrées mémoire documenté.

2. PLATEFORME DE TRAVAIL ET PRÉPARATION DE L'ENVIRONNEMENT

L'environnement SEED (suite SEED 2.0) réutilise le même dépôt Git. La machine virtuelle SEED Ubuntu 20.04 est l'environnement de référence; Docker et Docker Compose sont requis pour l'extension Serveur obligatoire (section 3, E9).

2.1 Vérification des outils

Commande	Ce qu'elle confirme
`gcc --version`, `gdb --version`, `objdump --version`	Le compilateur, le débogueur et le désassembleur sont présents.
`nasm -v` et `ld --version`	L'assembleur et l'éditeur de liens sont présents (développement de shellcode).
`gcc -m32 prog.c -o prog`	La compilation 32 bits fonctionne (bibliothèques multilib installées).
`sysctl kernel.randomize_va_space`	L'état de l'ASLR est lisible (0, 1 ou 2) et modifiable par `sudo sysctl -w`.
`gcc -fno-stack-protector -z execstack prog.c -o prog`	Les options de contre-mesure sont acceptées par le compilateur.
`gnatmake --version` (ou `alr version`)	La chaîne Ada/GNAT est disponible pour la partie D.

2.2 Contrôle des contre-mesures

Contre-mesure	Activer / Désactiver
Protection de dash	`sudo ln -sf /bin/zsh /bin/sh` (contournée) ou `sudo ln -sf /bin/dash /bin/sh` (active, à déjouer par `setuid(0)`).
ASLR	`sudo sysctl -w kernel.randomize_va_space=0` (désactivé) ou `=2` (actif).
StackGuard	`gcc -fno-stack-protector` (désactivé) ou compilation sans cette option (actif).
Pile non exécutable (NX)	`gcc -z noexecstack` (active la protection) ou `-z execstack` (pile exécutable).

2.3 Configuration de référence des protections par essai

Essai	ASLR	Pile (NX)	StackGuard et compilation
E3 Débordement et shellcode (T3)	Désactivé	Désactivée	Sans canari; `gcc -m32 -fno-stack-protector -z execstack` (stack.c, binaire Set-UID root).
E4 Contre-mesures (T4)	Varie	Varie	On réactive chaque protection à tour de rôle (dash, ASLR=2, -fstack-protector, -z noexecstack).
E5 Return-to-libc (T5)	Désactivé	Activée	Sans canari; `gcc -m32 -fno-stack-protector -z noexecstack` (retlib.c, binaire Set-UID root).

2.4 Isolement et sécurité

Toutes les manipulations restent dans l'environnement SEED isolé. Les binaires Set-UID sont compilés localement et ne sont jamais installés ailleurs. Les conteneurs restent sur un réseau Docker isolé. Scripts d'attaque et shellcodes demeurent dans le dépôt de l'équipe.

3. TRAVAIL EXIGÉ

Partie A — Shellcode et débordement de tampon (version Set-UID)

Id	Exigence	Critère d'acceptation
E1 E2 E3	Disposition de la pile, développement du shellcode, exploitation (Niveaux 1 à 3)	• Compiler `stack.c` en Set-UID root 32 bits (`gcc -m32 -z execstack -fno-stack-protector`, puis `chown root` et `chmod 4755`). Localiser le tampon, le pointeur de cadre et l'adresse de retour sous GDB. Calculer la distance exacte et dresser le schéma de la pile. • Assembler un shellcode 32 et 64 bits (`nasm`, `ld`) à partir des fichiers fournis (`hello.s`, `mysh64.s`, `another_sh64.s`), extraire le code machine et éliminer tous les octets nuls. Vérifier de façon autonome avec `call_shellcode.c`. • Compléter `exploit.py` pour produire `badfile` et obtenir un shell root sur `stack-L1` (Niveau 1). Réaliser le Niveau 2 (taille de tampon inconnue : plage 100-200 octets sans force brute) et le Niveau 3 (binaire 64 bits). Justifier chaque valeur.

Partie B — Contre-mesures et return-to-libc

Id	Exigence	Critère d'acceptation
E4	Contre-mesures et limites	Montrer ce que chaque protection bloque et ses limites : déjouer dash avec `setuid(0)` dans le shellcode; déjouer l'ASLR par force brute (`randomize_va_space=2`, `brute-force.sh`) sur stack-L1; réactiver StackGuard et NX, et expliquer pourquoi l'attaque de la partie A échoue.
E5	Return-to-libc	Sur `retlib.c` (compilé `-m32 -fno-stack-protector -z noexecstack`, Set-

Partie C — Outils de bas niveau (composante CLO 9)

Id	Exigence	Critère d'acceptation
E6	Chaîne source-assembleur-code machine	Pour la fonction vulnérable et le shellcode (E2), relier le code source, le désassemblage (`objdump -Mintel -d` , `xxd`) et le code machine; identifier le prologue, l'épilogue et l'instruction `ret` . Analyser l'effet d'au moins deux options de compilation ou contre-mesures (ex. `-fstack-protector` , `-O2` , `-z execstack` / `-z noexecstack` , `-m32`) sur l'assembleur et la faisabilité de l'attaque.

Partie D — Première défense par le typage (Ada)

Id	Exigence	Critère d'acceptation
E7	Réécriture en Ada	Réécrire la routine d'analyse d'entrée vulnérable en Ada avec sous-types contraints (chaîne de longueur bornée, index contraint). Rejouer la même entrée qu'en E3 et montrer que le contrôle de contrainte lève un `Constraint_Error` maîtrisé au lieu de déborder; traiter l'exception dans un état sûr sans corruption mémoire.
E8	Propriétés vérifiables et corpus légué	Écrire les hypothèses violées comme assertions ou propriétés explicites (longueur admissible, borne d'index, invariant du tampon) préparant les contrats SPARK. Constituer le corpus d'au moins huit entrées mémoire dangereuses documentées, légué au projet final.

Partie E — Extension obligatoire : exploitation à distance (version Serveur)

Id	Exigence	Critère d'acceptation
E9	Exploitation à distance (Serveur)	Rejouer l'attaque sur la version Serveur en conteneur : compiler et installer le serveur (`make` , `make install` ; 32 bits `-static`), démarrer les conteneurs (`dcbuild` , `dcup`), puis exploiter `stack.c` à distance par connexion TCP (port 9090) via `exploit.py` client. Documenter les différences avec l'attaque locale (absence d'accès au binaire, adresses, shell inversé).

4. EXIGENCES TECHNIQUES ET DE CODAGE

- **Environnement isolé :** Toutes les manipulations restent strictement dans l'environnement virtuel SEED ou conteneurs isolés.
- **Reproductibilité :** Chaque exploitation est fournie comme script rejouable avec sa trace. L'état de l'ASLR et les flags GCC doivent être explicités.
- **Code vulnérable et défense :** Les programmes C fournis ne sont pas modifiés. La routine Ada doit compiler et s'exécuter sans erreur.
- **Hypothèses écrites :** Chaque hypothèse de sécurité est formulée sous forme d'assertion explicite ou de précondition vérifiable.
- **Outils d'IA générative :** Permis comme outil d'appoint à condition d'être déclarés dans le rapport (outil, version, usage). Interdits lors de la démonstration.

5. ORGANISATION DU PROJET ET DÉPÔT GIT

- **Dépôt d'équipe :** Mettre à jour le dépôt Git avec les scripts d'attaque, le shellcode, la routine Ada, les traces et le fichier `JOURNAL.md` .
- **Rotation des rôles :** Inverser les rôles réalisation/validation par rapport au laboratoire précédent. Consigner la rotation dans le journal.
- **Corpus :** Organiser le dossier `corpus/` avec au moins 8 entrées mémoire documentées (une par fichier ou par ligne).

6. DÉROULEMENT DES SÉANCES

Séance	Activités	Résultat attendu
Séance 1	Disposition de la pile (E1); shellcode (E2); débordement N1-N3 (E3); contre-mesures (E4). Relevés objdump pour E6.	E1 à E4 réalisés; shellcode sans octet nul; exploit rejouable; effet des contre-mesures observé.
Entre-séances	Finaliser l'analyse source-assembleur-code machine et l'effet des options de compilation (E6). Préparer les conteneurs Serveur.	Analyse L2-B rédigée, prête pour vérification.
Séance 2 & Fin de labo	Return-to-libc (E5); exploitation Serveur (E9); vérification E6; réécriture Ada (E7); propriétés/corpus (E8); démonstration devant l'assistant. Finalisation du rapport et remise.	Exigences démontrées; `Constraint_Error` observé; corpus constitué; démonstration validée; rapport remis avant l'échéance.

7. ESSAIS ET TRACES EXIGÉS

Le rapport présente un tableau des essais reprenant au minimum les cas ci-dessous, avec pour chacun la preuve (trace, capture, extrait) et l'état des contre-mesures.

Essai	Procédure	Résultat attendu	Preuve
T1 Disposition pile	Localiser tampon, ebp, adresse de retour; calculer distance (E1).	Distance exacte obtenue; disposition documentée.	Trace GDB/objdump et schéma.
T2 Shellcode	Assembler et exécuter de façon autonome (E2).	`/bin/sh` obtenu (a32/a64); 0 octet nul.	
T3 Débordement	Produire badfile et attaquer stack-L1 à L3 (E3).	Shell root obtenu; charge unique (N2); 64 bits (N3).	Scripts rejouables et traces id.
T4 Contre-mesures	Déjouer dash, forcer ASLR, réactiver StackGuard & NX (E4).	Effet et limites de chaque contre-mesure montrés.	Traces avant/après, sysctl, flags GCC.
T5 Return-to-libc	Compléter exploit.py (X,Y,Z) et attaquer retlib.c (E5).	`system("/bin/sh")` exécuté sur pile NX active.	Script, adresses, trace terminal.
T6 Source-ASM (L2-B)	Relier C/ASM/code machine; comparer options GCC (E6).	Prologue, epilogue, ret identifiés; options analysées.	objdump annoté, tableau comparatif.
T7 Défense Ada	Rejouer l'entrée de T3 sur la routine écrite en Ada (E7).	`Constraint_Error` levé et traité; pas de débordement.	Source Ada et trace d'exécution.
T8 Serveur distant	Exploiter stack.c à distance par TCP en conteneur (E9).	Shell distant ou inversé obtenu (port 9090).	exploit.py client + trace conteneur.

8. LIVRABLES ET STRUCTURE DU RAPPORT

Livrable	Contenu et Format
1. Code et scripts	`stack.c`, `exploit.py` (E3); `hello.s`, `mysh64.s`, `convert.py`, `call_shellcode.c` (E2); `retlib.c` et son `exploit.py` (E5); routine Ada et projet `.gpr` (E7). (Dossier dans l'archive).
2. Rapport principal	Format PDF rédigé selon la structure de la section 8.1.
3. Composante L2-B	Analyse de la chaîne source-assembleur-code machine et effet des options de compilation (Section dédiée ou document séparé).
4. Journal & Dépôt	Dépôt Git actif + `JOURNAL.md` (rôles, rotation, décisions).
5. Corpus mémoire	Dossier `corpus/` contenant au moins 8 entrées documentées.
6. Extension Serveur	Fichiers `server-code`, `attack-code`, `docker-compose.yml` et traces d'exploitation à distance (E9).
7. Traces & Déclaration	Captures/traces des essais T1 à T8 + Déclaration d'usage des outils d'IA générative.

8.1 Structure exigée du rapport (PDF)

1. **Introduction et modèle de menace :** Objectif, programmes vulnérables, actifs protégés et surfaces d'entrée.
2. **Disposition de la pile et contre-mesures :** Cadre d'appel, distance au tampon (E1), effet de dash, ASLR, StackGuard et NX (E4).
3. **Exploitations mémoire :** Shellcode (E2), débordement N1-N3 (E3), return-to-libc (E5); cause première, traces et propriétés violées.
4. **Outils de bas niveau (L2-B) :** Chaîne source-assembleur-code machine, options de compilation et limites (E6).
5. **Défense par le typage :** Réécriture Ada, contrôle de contrainte (E7), assertions (E8), essai T7.
6. **Corpus légué et limites :** Contenu du corpus, garanties et limites du typage et des contre-mesures.
7. **Extension à distance (Serveur) :** Exploitation en conteneur par TCP (E9) et comparaison avec l'attaque locale.
8. **Travail d'équipe :** Rôles par séance, décisions, contribution de chacun (référence à l'historique Git).
9. **Déclaration :** Sources réutilisées, usage des outils d'IA générative; annexes (code commenté, traces).

9. DÉMONSTRATION DEVANT L'ASSISTANT

• **Organisation :** Présence obligatoire de tous les membres lors de la séance 2. Déroulement sur 20 minutes (15 min d'essais, 5 min de questions individuelles). Démontrés en direct : T3, T5, T8 et T7.
• **Questions individuelles :** Au moins deux questions par membre sur une partie tirée au sort. La note de la partie A est individualisée (critère A6). Une absence non justifiée entraîne un zéro pour la partie A (30 points).

10. GRILLE D'ÉVALUATION DÉTAILLÉE (100 POINTS = 5% DE LA NOTE FINALE)

Section / Critères	Pts	Niveaux d'évaluation (Maîtrisé 100% | Satisfaisant 70% | Partiel 40% | Non démontré 0%)
A. Démonstration en direct (Note individualisable)	**30**	• **A1 Shellcode & débordement (7 pts) :** Distance exacte, shellcode 0 octet nul, exploit root N1-N3 direct. • **A2 Contre-mesures (5 pts) :** dash, ASLR, StackGuard, NX démontrés et expliqués. • **A3 Return-to-libc (6 pts) :** NX contourné par system(), offsets X,Y,Z expliqués. • **A4 Attaque distante (4 pts) :** Serveur conteneur exploité par TCP, différences expliquées. • **A5 Défense Ada (5 pts) :** Entrée inerte, Constraint_Error levé et traité. • **A6 Questions individuelles (3 pts) :** Maîtrise individuelle de la pile, du shellcode et d'Ada.
B. Attaques mémoire & bas niveau	**30**	• **B1 Disposition pile & protections (7 pts) :** Cadre d'appel exact, limites des protections expliquées. • **B2 Exploitations reproductibles (13 pts) :** Scripts E2, E3, E5, E9 rejouables avec causes premières. • **B3 Chaîne source-ASM-machine (10 pts) :** Correspondance C/ASM/Machine, options GCC analysées (L2-B).
C. Défense par le typage & code	**25**	• **C1 Réécriture Ada (10 pts) :** Sous-types contraints, exception gérée dans un état sûr. • **C2 Propriétés vérifiables (8 pts) :** Hypothèses écrites comme assertions (bornes, invariants). • **C3 Qualité code/scripts (7 pts) :** Code lisible, commenté, compilant sans warning.
D. Rapport, traces & travail d'équipe	**15**	• **D1 Rapport & traces T1-T8 (6 pts) :** Structure complète, preuves d'essais complètes. • **D2 Corpus mémoire légué (4 pts) :** Au moins 8 entrées dangereuses documentées. • **D3 Dépôt Git & journal (5 pts) :** Contributions des 2 membres, rotation des rôles, `JOURNAL.md`.

PÉNALITÉS ET RÈGLES DE CALCUL

Situation	Conséquence
Remise en retard	Note de zéro pour le laboratoire (sauf prolongation autorisée ou motif médical justifié).
Absence non justifiée à la démo	Note de zéro pour la partie A (30 points) pour le membre absent.
Attaque hors environnement isolé	Traitée comme une inconduite académique grave; note du laboratoire compromise.
Exploitation non rejouable	Critère correspondant limité à ce qui est démontré en séance; scripts non rejouables non crédités.
Incapacité d'expliquer le travail / IA non déclarée	Critère A6 à zéro et partie A réduite de moitié; traitement selon les règlements d'intégrité académique.

11. RÉFÉRENCES PRINCIPALES ET RESSOURCES

- **Manuel de référence :** Wenliang Du, *Computer & Internet Security: A Hands-on Approach*, 3e éd., 2022. Chapitres 4 (Buffer Overflow), 5 (Return-to-libc) et 9 (Shellcode Development).
- **Documentation Ada :** AdaCore, *Introduction to Ada*. Chapitres « Strongly typed language », « Records » et « Arrays ».
- **Environnement SEED Labs :** SEED Security Labs (suite SEED 2.0) — Thèmes *Shellcode Development*, *Buffer Overflow (Set-UID & Server)*, *Return-to-libc*.
- **Normes de codage :** SEI CERT C Coding Standard (Règles ARR, STR, MEM). OWASP & NIST Cybersecurity Framework 2.0.